

AUFGABE ZUR VORLESUNG

Übung 07 – Capstone: der komplette Stack

Modul	3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema	Docker: alles zusammen – Python-App, Datenbank, Secrets, Compose
Dozent	Dr. Ulrich Winkler
Semester	5. Semester · Informationstechnik

Vorlesung: Decks *docker-basics*, *docker-data*, *docker-network*, *docker-compose* **Ziel:** Alles Gelernte zu einem funktionierenden Gesamtsystem zusammenführen: ein Python-Werkzeug, das über ein Docker-Netzwerk mit einer PostgreSQL-Datenbank spricht, deklarativ per Compose gestartet, mit Passwort als Secret und Daten in einem Volume. Beobachtet über **Portainer** und **Adminer**.

Kontext

Dies ist die Zusammenfassung des ganzen Kurses – und zugleich der Aufbau, der Ihnen in den kommenden **Python-Kursen** wieder begegnet: eine Anwendung plus eine Datenbank, sauber in Container verpackt. Alle Bausteine kennen Sie bereits:

- eigenes Image aus einem `Dockerfile` (Übung 03),
- Datenbank + Netzwerk (Übung 04),
- Compose-Stack (Übung 05),
- Passwort als Secret (Übung 06),
- Datenpersistenz über ein Volume (Übung 02).

Portainer läuft bereits aus der Dev-Container-Einrichtung (Port 9000) – nutzen Sie es als Kontrollfenster.

Aufgabenstellung

1. Legen Sie die Secret-Datei an (`db_password.txt` aus der Vorlage) und starten Sie den kompletten Stack mit Compose.
2. Prüfen Sie in **Portainer**, dass Container, Netzwerk und Volume existieren.
3. Schreiben Sie mit `log_tool` mehrere Einträge und listen Sie sie (`add` / `list`).
4. Kontrollieren Sie dieselben Einträge in **Adminer** (Tabelle `logs`).
5. Weisen Sie die **Persistenz** nach: `down` (ohne `-v`), dann erneut `up` – die Einträge sind noch da. Räumen Sie am Ende vollständig auf (`down -v`).

Tipp: Wenn `log_tool` die Datenbank „noch nicht bereit“ meldet, ist Postgres beim allerersten Start noch am Initialisieren. Einfach den `log_tool` -Aufruf wiederholen.

Musterlösung

```
cd uebungen/07_capstone

# 1) Secret anlegen und Stack starten
cp db_password.txt.example db_password.txt
```

```
docker compose up -d --build
docker compose ps

# 2) In Portainer (Port 9000): Container (db, adminer), Netzwerk und
#     Volume pgdaten sichtbar.

# 3) Einträge schreiben und lesen
docker compose run --rm log_tool add "Projektstart"
docker compose run --rm log_tool add "Datenbank steht"
docker compose run --rm log_tool add "Alles läuft im Container"
docker compose run --rm log_tool list
#   1 2026-... Projektstart
#   2 2026-... Datenbank steht
#   3 2026-... Alles läuft im Container

# 4) In Adminer (Port 8080): Server=db, Datenbank=logbuch, Tabelle "logs"
#     -> dieselben drei Zeilen.

# 5) Persistenz nachweisen
docker compose down          # Container weg, Volume bleibt
docker compose up -d
docker compose run --rm log_tool list    # Einträge sind noch da

# Vollständig aufräumen (auch die Daten):
docker compose down -v
```

Was Sie erreicht haben: Ein vollständiger, reproduzierbarer Mehr-Container-Stack aus Python-Anwendung und Datenbank – gestartet mit einem einzigen Befehl, konfiguriert über Environment, abgesichert per Secret, mit persistenten Daten. Auf genau diesem Muster bauen die kommenden Python-Kurse auf.